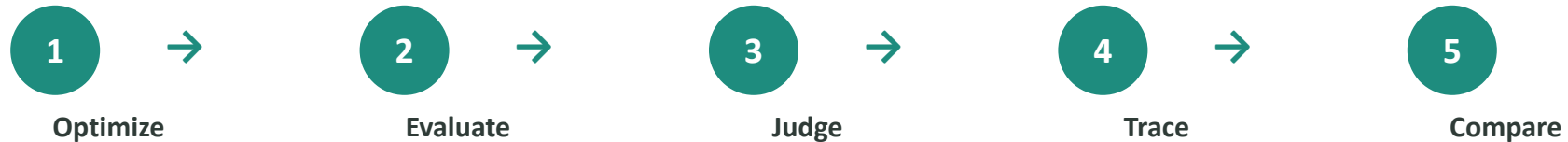


Make RAG Better, Then Prove It

Sharper retrieval, real evaluation scores, and the tracing tools (LangSmith + Langfuse) that show you what your bot is actually doing.



Learn with Sarvesh · Courses. Clarity. Confidence.

What You Already Have

Last week, in one live session, you built all of this:

1

RAG Pipeline

Load → chunk → embed → retrieve →
answer

2

Chunking

Splitting a PDF into searchable pieces

3

Embeddings & Store

Text → vectors, saved in a vector DB

4

Retrieval

Pulling the top matching chunks

You have a bot that works. Today we make it **RELIABLE**.



REMEMBER THIS: A working demo and a production-ready RAG system are not the same thing — today's session is the gap between them.

PART 1

Why “It Works” Isn’t Enough

Your bot answers questions. But does it answer them well, every single time?

Why this matters: half the RAG systems that impress in a demo start missing obvious answers the moment a real user phrases things their own way.

The Problem With Naive Retrieval

1

One phrasing, one shot

Plain similarity search only searches the words you typed — not the words the document actually uses.

2

Noisy top-K

The top matches often include half-relevant chunks that just add clutter for the LLM to wade through.

3

No feedback loop

Without a score, you can't tell if yesterday's tweak made retrieval better or quietly made it worse.



REMEMBER THIS: Naive RAG isn't broken — it's brittle. A small wording difference between the user's question and the document's language can break the whole answer.

A Familiar Support-Desk Problem

A customer asks about their policy — but never in the words the document uses:

1

"I paid late"

Document says: grace period

2

"My claim got denied"

Document says: exclusion clause

3

"Extra covers I bought"

Document says: riders / add-ons

One retriever call, one phrasing. **Miss the wording, miss the answer.**

 **ASK THE ROOM:** Think of one question your users ask in at least three different ways. That exact gap is what breaks single-query retrieval.

Two Fixes for Sharper Retrieval

Both fixes slot into your existing retriever — no rebuild required:



Multi-Query Retriever

Ask the same question several different ways, search each one, then merge the results.



Contextual Compression

After retrieving, strip each chunk down to only the sentences that actually answer the question.

Simply: **Ask it more ways. Read only what matters.**



WRITE THIS DOWN: Multi-Query fixes what you search for. Contextual Compression fixes what survives after you've found it. Most real systems use both.

Multi-Query Retriever, Step by Step

One question becomes several — then the results get merged into one clean set:



"What happens if I pay late?" → *"grace period", "late premium penalty", "missed due date consequences"*



WRITE THIS DOWN: Multi-Query = ask it several ways, search all of them, then merge and de-duplicate. More recall, same document.

Contextual Compression, Step by Step

Retrieve as usual — then trim every chunk down to just what answers the question:



→ LLM → gets distracted by irrelevant text in context window;

A 400-word policy clause becomes the **one sentence** that actually answers the question.

↳ ("lost in the middle")

 **INTERVIEW Q:** "Why not just retrieve fewer chunks instead of compressing?" → Fewer chunks risks missing the answer entirely; compression keeps the recall but removes the noise after the fact.

PART 2

Measuring What “Good” Means

You can't fix what you can't measure — an answer can feel right and still be wrong.

Why this matters: in an interview or on the job, “it looks fine to me” is not an evaluation strategy. Real metrics are what separate a toy project from something a team can trust.

Why Evaluation Actually Matters

1

Scale

You can't personally read ten thousand answers before every release.

2

Regression

Did last night's prompt tweak help or quietly hurt?
Only a metric tells you.

3

Trust

Stakeholders want numbers before they'll let you ship to real users.



REMEMBER THIS: Evaluation isn't extra work on top of the project — it IS the project, once you're past the demo stage.

Faithfulness vs. Relevance

The two questions every RAG answer needs to survive:

(↑)F (↓)R → correct facts
wrong question answered



(↓)F (↑)R → Made up the details



Faithfulness

Is the answer actually supported by the retrieved context — or did the model add something that isn't there?

+



Relevance

Does the answer actually address what was asked — even if every word in it happens to be true?

Faithfulness = no hallucination. **Relevance = actually answers the question.**



WRITE THIS DOWN: Classic example: ask "what is the capital of France?" and get back "Paris has a population of about 2 million." True, faithful, and completely irrelevant.

Combining the Scores

One question and answer pair, scored two different ways, then combined:



 **REMEMBER THIS:** A high faithfulness score with a low relevance score usually means one thing: correct answer, wrong question. Look at both numbers, never just one.

LLM-as-a-Judge

(↑) Super Big model ⇒ Short rubric score → (1 to 5)

↳ scale at

1

What it is

A strong LLM grades another model's answer against a rubric, like a teacher marking an exam.

Judges ⇒ Praises

2

Why use it

Far cheaper and faster than a human reviewing every answer — scales to thousands of cases.

⇒ use Judge for Bulk of your evaluation

3

Watch out for

Judges can favor longer, more confident-sounding answers. Spot-check against real humans.

Randomly Judge ⇒ Human agree



INTERVIEW Q: "How do you evaluate a RAG system when you don't have ground-truth labels?" → LLM-as-a-judge with a clear rubric is a standard, defensible answer.

The Judge in Action

Question: "How do I reset my password?" — two candidate answers, one judge:



Answer A

"Click Forgot Password on the login page, check your email for a link, and set a new password."

VS



Answer B

"Passwords can be reset through the account settings menu if you are logged in, or via support if not."

Judge verdict: Answer A wins — more direct, fewer conditional branches, matches how most users actually reset a password.



WRITE THIS DOWN: Direct, pairwise comparisons ("which answer is better") are often more reliable than asking a judge for an absolute 1-5 score.

You Don't Have to Hand-Roll This

1

Same idea, maintained for you

Faithfulness and Relevance -- the exact metrics you just hand-built -- as tested, versioned library code instead of a prompt only you maintain.

2

RAGAS — the RAG specialist

Started as a research paper in 2023, laser-focused on RAG: Faithfulness, Response Relevancy, Context Precision/Recall -- reference-free and light to set up.

3

DeepEval — the full test suite

50+ metrics across RAG, agents, and safety, built “pytest for LLMs” style -- evaluations become CI-gated tests, not one-off notebook cells.



REMEMBER THIS: Your hand-rolled judge and these libraries are asking the exact same question -- understanding the mechanics first is what lets you debug a library score that looks wrong.

RAGAS vs. DeepEval, Head to Head

RAGAS

- Born as a research paper in 2023, cited by OpenAI at DevDay
- Laser-focused: Faithfulness, Response Relevancy, Context Precision/Recall
- Reference-free -- most metrics need no ground-truth labels
- Lightweight -- a few lines to score a RAG pipeline
- Best when RAG metrics are the whole job

DeepEval

- Built by Confident AI, “pytest for LLMs”
- 50+ metrics: RAG, agents, chatbots, safety (bias, toxicity, hallucination)
- Custom criteria via G-Eval, plus assertions and thresholds
- CI/CD-native -- evaluations run as gated tests, not notebook cells
- Optional Confident AI cloud for team collaboration & monitoring



REMEMBER THIS: Rule of thumb: RAG metrics are the whole job and you want minimal setup → RAGAS. You want CI-gated tests, custom criteria, or coverage beyond RAG (agents, safety) → DeepEval.

PART 3

Seeing Inside the Black Box

Your chain runs in milliseconds. When it's wrong, you need to see exactly which step failed.

Why this matters: “it's not working” is not a bug report. A trace that shows the exact prompt, the retrieved chunks, and the raw model output is.

What Is LLM Observability?

1

Trace every call

See the exact prompt, retrieved chunks, and output for every single request.

2

Debug fast

Find the failing step in seconds instead of guessing and re-running blind.

3

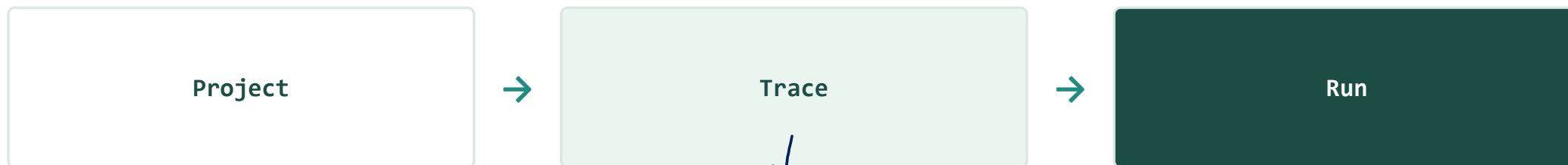
Prove quality over time

Track scores and latency across versions, not just at launch.

 **REMEMBER THIS:** Observability is what evaluation looks like once your bot is live and talking to real users, not just your test set.

LangSmith's Core Concepts

Three words you'll see constantly once you open the LangSmith dashboard:



Customer ⇒ Restaurant

- Project = your whole app (e.g. “Insurance Support Bot”).
- Trace = one full user request, start to finish.
- Run = one step inside that trace — a retrieval call, an LLM call, a tool call.

Walking IN

Walking OUT

→ Seat → R1
 → Order → R2
 → Handwash → R3

every single action

Question

Answer

Trace

→ Run ⇒ One single action within that visit

Two Ways to Get Traces

@traceable
↓
decorator

@traceable
def



Automatic (LangChain)

If you're already using LangChain, tracing is often just one environment variable away — every chain and retriever call gets captured for free.

Automatic =>



Manual (@traceable)

For your own custom Python functions — outside LangChain — wrap them with the @traceable decorator to log inputs, outputs, and timing.

or



REMEMBER THIS: You don't have to choose one — most real projects mix both: automatic tracing for the LangChain parts, @traceable for the custom glue code around it.

Datasets, Evaluators & Feedback

1

Datasets

A saved set of test questions you re-run every time you change something.

2

Custom evaluators

Your own faithfulness/relevance scoring functions, run automatically on every trace.

3

Feedback scoring

Attach a thumbs-up/down or a numeric score directly onto a specific run.

! **REMEMBER THIS:** A dataset plus an evaluator turns “did I break anything” from a guess into a one-click re-run before every deploy.

Comparing Runs & the Prompt Hub



Compare Runs

Put two versions of your pipeline side by side — same questions, different prompts or settings — and see exactly where scores moved.



Prompt Hub

A shared library of reusable, versioned prompts — so your best prompt isn't buried in one notebook cell.



INTERVIEW Q: "How do you know a prompt change actually improved things?" → Run both versions against the same dataset in LangSmith and compare the scores side by side, don't just eyeball a few examples.

PART 4

LangSmith vs. Langfuse

Same job — watch every LLM call — but two very different companies behind the tool.

Why this matters: you will get asked “which observability tool have you used” in interviews. Knowing both, and why you'd pick one over the other, is the actual signal.

What Is Langfuse?

langsmith

- ① Tracing
- ② Evaluation
- ③ Datasets
- ④ Prompt management

1

Open-source core

The core tracing and eval engine is open source — you can read the code, self host it, or use their cloud.

2

Framework-agnostic

Works with LangChain, LlamaIndex, or plain API calls — it doesn't assume you're using any particular framework.

3

Tracing + evals + prompts

Covers the same ground as LangSmith: traces, datasets, evaluators, and prompt management.

llamaindex / Haystack / crewai / Antronic / OpenAI

langfuse

=>

Open-Source

=>

Banking / Ins / Healthcare

=>

Sit in own-server



REMEMBER THIS: Langfuse being open-source doesn't just mean it's free — it means a company can run it entirely inside their own infrastructure, which matters a lot for regulated industries like insurance or healthcare.

LangSmith vs. Langfuse, Head to Head

LangSmith

- Built by the LangChain team
- Deepest native fit for LangChain / LangGraph
- Managed SaaS, with a free tier
- Strong Prompt Hub for shared prompts
- Pricing scales with trace volume

Langfuse

- Open-source core, self-host or cloud
- Framework-agnostic — LangChain, LlamaIndex, raw API
- Free if self-hosted; usage-based cloud tier
- Strong on cost & latency dashboards
- Growing eval + prompt management features

 **REMEMBER THIS:** Rule of thumb: all-in on LangChain and want the path of least resistance → LangSmith. Need self-hosting, data residency, or you're framework-agnostic → Langfuse.

PART 5

Hands-On: Build It Yourself

Time to open the notebook — reading about multi-query retrieval and watching it fix a bad answer are not the same thing.

Why this matters: the fastest way to actually own these concepts for an interview is to have broken them yourself, at least once, in your own notebook.

Honest Limitations & What's Next

HONEST LIMITATIONS

- LLM judges can disagree with human reviewers — spot-check regularly.
- Judges and tracing both cost extra tokens and latency.
- More tools (LangSmith + Langfuse) means more setup and upkeep.
- None of this fixes a genuinely bad or missing document.

WHERE YOU GO NEXT

1

RAGAS framework

a standardized library of RAG metrics, built on the same ideas as today

2

Cost-aware evaluation

tracking answer quality per rupee/dollar spent, not quality alone

3

Production monitoring

live alerts when faithfulness or relevance drops on real traffic



REMEMBER THIS: RAG is powerful but not magic — its answer quality is only as good as the data, the retrieval, and the evaluation behind it.



See You in Week 7!

Today you made RAG sharper AND learned to prove it with real metrics — and you can now speak confidently about LangSmith and Langfuse in any interview. Keep tracing your own experiments.

[Learn with Sarvesh](#) · [Courses. Clarity. Confidence.](#)